

CUSTOMER TICKET API

Integrated DevOps / CI-CD Project Documentation

Project Type	Hands-on DevOps portfolio project
Implemented Stack	Python + Git/GitHub + Jenkins + Docker + Kubernetes
Primary Goal	Automate application build, containerization and Kubernetes deployment
Repository	github.com/bhattra hul76660-cyber/cloud-support-devops-project
Kubernetes Environment	Minikube (Docker driver), Kubernetes v1.31.0
Application Port	5000 (NodePort exposed on 30080)

Implemented project record - based on the working setup and verification completed during the project session.

Project Status: Core implementation completed

1. Project Overview

This project demonstrates a practical CI/CD workflow for a small Python Customer Ticket API. The idea is simple: when a developer changes application code and pushes it to GitHub, the DevOps pipeline builds a new Docker image and deploys the application toward Kubernetes. The project was intentionally kept at a beginner-to-junior DevOps level so the complete flow could be understood and demonstrated.

Real project scenario: A customer-support application exposes an API. Developers should not manually rebuild containers and redeploy the application after every code change. The DevOps process should automate those repetitive steps.

The implemented flow was:

Developer -> GitHub -> Jenkins -> Docker -> Kubernetes -> Customer Ticket API -> Browser/User

Important implementation note: The Jenkins pipeline successfully built the Docker image and executed the Kubernetes deployment command. Because the local Jenkins Docker environment and Minikube image environment were separate, the final verification also used **minikube image load** manually before restarting the Kubernetes deployment. Therefore, this project should be described as an integrated CI/CD demonstration with a manual image-transfer step for the local Minikube lab, not as a fully automatic production image-registry deployment.

2. Technologies Used

Technology	Purpose in the Project
Python / Flask-style API	Provides the Customer Ticket API application.
Git	Tracks source-code changes locally.
GitHub	Remote repository and source-of-truth for the application code.
Jenkins	Runs the CI/CD pipeline and automates checkout, Docker build/run and Kubernetes deployment commands.
Docker	Packages the application into a runnable container image.
Kubernetes / Minikube	Runs and manages the application as pods behind a Service.
PowerShell / Windows	Local command-line environment used to operate the lab.

The project combines the tools already practiced separately into one connected workflow. The value is not learning more tools; it is proving that the tools can work together.

3. Architecture and Workflow

The architecture is a simple source-to-deployment chain. Each component has one clear responsibility.

Stage	Component	What happens
1	Developer	Changes app.py or other project files.
2	Git / GitHub	Changes are committed and pushed to the main branch.
3	Jenkins	Checks out the repository and runs the pipeline stages.
4	Docker	Builds customer-ticket-api:latest and runs the container.
5	Kubernetes	Updates the deployment and waits for rollout completion.
6	Service / Browser	The NodePort service exposes the API so the updated response can be verified.

Result: the same application code moves through source control, automation, packaging and orchestration instead of being rebuilt manually at every step.

4. Jenkins Pipeline

The Jenkins job created for this project is **customer-ticket-api-pipeline**. It is a Pipeline job using the repository above and the **main** branch. The Jenkinsfile was stored in the repository and used as the pipeline definition.

The final Jenkinsfile used four main stages:

Stage	Purpose	Key action
Checkout	Get source code	git branch: main + repository URL
Docker Build	Create application image	docker build -t customer-ticket-api:latest .
Docker Run	Run new container version	Stop/remove old container, then docker run -d ... -p 5000:5000
Deploy to Kubernetes	Update K8s deployment	kubectl set image ... followed by rollout status

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
        url: 'https://github.com/bhattraahul76660-cyber/cloud-support-devops-project.git'
      }
    }

    stage('Docker Build') {
      steps {
        sh 'docker build -t customer-ticket-api:latest .'
      }
    }

    stage('Docker Run') {
      steps {
        sh 'docker stop customer-ticket-api || true'
        sh 'docker rm customer-ticket-api || true'
        sh 'docker run -d --name customer-ticket-api -p 5000:5000 customer-ticket-api:latest'
      }
    }

    stage('Deploy to Kubernetes') {
      steps {
        sh '''
        export KUBECONFIG=/tmp/kubeconfig
        kubectl set image deployment/customer-ticket-api customer-ticket-api=customer-ticket-api:latest
        kubectl rollout status deployment/customer-ticket-api
        '''
      }
    }
  }
}
```

Why sh instead of bat? Jenkins was running inside a Linux-based Jenkins Docker container, so Linux shell commands were required. The earlier Windows **bat** commands failed in that environment; switching to **sh** fixed the pipeline execution.

5. Docker and Jenkins Setup

To allow Jenkins to run Docker commands, a custom Jenkins image was created from the official Jenkins LTS JDK 21 image. Docker CLI was installed inside that image.

```
FROM jenkins/jenkins:lts-jdk21

USER root

RUN apt-get update && \
    apt-get install -y docker.io && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

USER jenkins
```

The custom image was built as **custom-jenkins:1.0**. The running Jenkins container was named **Jenkins** and used the Docker socket so Jenkins could invoke Docker on the host environment.

```
docker run -d --name Jenkins --restart=on-failure --user root -p 8080:8080 -p 50000:50000 -v
jenkins_home:/var/jenkins_home -v //var/run/docker.sock:/var/run/docker.sock custom-jenkins:1.0
```

The setup was verified by running Docker commands from inside the Jenkins container. Docker CLI returned version **26.1.5**, and **docker ps** worked successfully.

Jenkins persistence: Jenkins home was mounted to the named Docker volume **jenkins_home**, which preserves the Jenkins configuration and jobs across container recreation.

6. Kubernetes Deployment

The application was deployed to a local Minikube Kubernetes cluster using the Docker driver. The cluster was verified as ready and the minikube node was in the **Ready** state.

Resource	Observed configuration
Cluster	Minikube, Kubernetes v1.31.0
Deployment	customer-ticket-api
Replicas	2 pods ready
Service	customer-ticket-service
Service type	NodePort
Service port	5000:30080/TCP
Minikube IP	192.168.49.2

```
kubectl get nodes
kubectl get pods
kubectl get svc
minikube service customer-ticket-service --url
```

The application was successfully opened in the browser through the Minikube Service URL. Two application pods were running, demonstrating basic Kubernetes replica management.

7. Jenkins -> Kubernetes Integration

Jenkins needed Kubernetes credentials that were valid from inside the Jenkins container. The host kubeconfig originally referenced local Windows certificate paths and a localhost API endpoint, which could not be used directly from the Jenkins container.

The working integration used the Minikube API endpoint reachable from the Jenkins container and copied the required certificate files into the container. A dedicated kubeconfig was created at **/tmp/kubeconfig**.

```
docker cp "$HOME\.minikube\ca.crt" Jenkins:/tmp/ca.crt
docker cp "$HOME\.minikube\profiles\minikube\client.crt" Jenkins:/tmp/client.crt
docker cp "$HOME\.minikube\profiles\minikube\client.key" Jenkins:/tmp/client.key
```

The resulting kubeconfig pointed to **https://192.168.49.2:8443** and used the copied CA, client certificate and client key.

```
docker exec Jenkins sh -c "KUBECONFIG=/tmp/kubeconfig kubectl get nodes"
```

This command returned the Minikube node in the Ready state, proving that Jenkins could authenticate to and query the Kubernetes cluster successfully.

Verification outcome: the Jenkins container had both Docker access and Kubernetes access, which are the two core permissions needed for this local CI/CD lab.

8. Real CI/CD Verification

The project was not treated as complete until a real application change was tested from source code to the browser.

Test	Observed result
Initial application response	Customer Ticket API is running
Code change	Response changed to: Customer Ticket API v2 is running
Git operation	Committed update to app.py and pushed successfully to main
Jenkins	Pipeline completed successfully
Docker	New customer-ticket-api:latest image built and container run
Kubernetes	Deployment rollout command executed successfully
Final verification	Browser showed the updated v2 response after Minikube image load and rollout restart

For the local Minikube environment, the image transfer step was performed with:

```
minikube image load customer-ticket-api:latest
kubectl rollout restart deployment/customer-ticket-api
kubectl rollout status deployment/customer-ticket-api
```

What this proves: a small source-code change can travel through version control, CI automation, containerization and Kubernetes deployment, and the user-facing application can be verified at the end of the process.

9. Important Commands Used

Purpose	Command
Check Kubernetes node	kubectl get nodes
Check pods	kubectl get pods
Check services	kubectl get svc
Get service URL	minikube service customer-ticket-service --url
Load local Docker image to Minikube	minikube image load customer-ticket-api:latest
Restart K8s deployment	kubectl rollout restart deployment/customer-ticket-api
Check rollout	kubectl rollout status deployment/customer-ticket-api
Read Jenkins initial password	docker exec Jenkins cat /var/jenkins_home/secrets/initialAdminPassword
Check Docker inside Jenkins	docker exec Jenkins docker ps
Check K8s inside Jenkins	docker exec Jenkins sh -c "KUBECONFIG=/tmp/kubeconfig kubectl get nodes"

These commands were part of the working troubleshooting and verification process, not just theoretical examples.

10. What This Project Demonstrates

This project demonstrates the ability to connect the following concepts:

Skill	Practical proof
Git/GitHub	Commit and push application changes to main branch.
Jenkins / CI-CD	Pipeline executes source checkout, build, run and deployment stages.
Docker	Build and run an application container from the Jenkins environment.
Kubernetes	Run two replicas behind a NodePort Service and perform a rollout.
Troubleshooting	Resolved Jenkins shell mismatch, Git checkout issues and Jenkins-to-Kubernetes connectivity/authentication.

Important learning point: DevOps is not only about knowing commands individually. The stronger demonstration is knowing where each tool fits in the end-to-end delivery process.

What was intentionally not added: a container registry, cloud Kubernetes cluster, webhook-triggered production deployment, secrets manager or full observability stack. Those are logical next-step production improvements, but they were not required for this beginner/junior portfolio project.

11. Interview Explanation

A clear 30-45 second explanation of the project:

"I built a Customer Ticket API deployment pipeline using Python, GitHub, Jenkins, Docker and Kubernetes. The application source code is stored in GitHub. Jenkins checks out the main branch, builds a Docker image, runs the container and updates the Kubernetes deployment. I used Minikube for the local Kubernetes environment and configured Jenkins with Kubernetes access. I verified the flow by changing the API response from version 1 to version 2, pushing the code to GitHub, running the Jenkins pipeline and then verifying the updated response in the browser. In the local lab, I manually loaded the new image into Minikube because the Jenkins Docker environment and Minikube image environment were separate."

Interview one-liner: "Automate the build, containerization and Kubernetes deployment of a Customer Ticket API whenever application code changes."

Common follow-up questions to prepare for: Why Docker? Why Kubernetes? Why Jenkins? What is a deployment vs a pod? What is a Service? Why did bat fail and sh work? Why was minikube image load needed? How did Jenkins authenticate to Kubernetes?

12. Final Project Status

Area	Status	Evidence / note
Python application	Completed	Customer Ticket API runs and returns expected response.
Git/GitHub	Completed	Code committed and pushed to main.
Jenkins pipeline	Completed	Pipeline job runs successfully.
Docker integration	Completed	Custom Jenkins image has Docker CLI/socket access; app image built and run.
Kubernetes	Completed	Deployment with two pods and NodePort Service verified.
Jenkins -> Kubernetes auth	Completed	kubectrl get nodes works from inside Jenkins using /tmp/kubeconfig.
Real update test	Completed	v1 -> v2 response verified in browser.
Documentation	Completed	This document. Screenshots/PPT can be added separately for portfolio presentation.

The core project is complete as a local integrated DevOps demonstration. For a stronger production-style version later, the next major enhancement would be adding a container registry and moving from manual Minikube image loading to an automated image push/pull flow.

END OF DOCUMENT